MACHINE LEARNING WITH PYTHON

Detailed Class-by-Class Covered Topics & Code References

Class 01: Foundations of Data Ingestion & Structural Inspection

Core Notebook Reference: ML_Class03.ipynb (Titanic Dataset Baseline)

- **Core Environment Setup:** Data handling ebong visual analysis er jonno structured analytics environment ready kora.

```
import pandas as pd
import matplotlib.pyplot as plt
```

- **Data Ingestion Baseline:** `pd.read_csv()` function babohar kore tabular data processing module set kora.

```
dataset = pd.read_csv('titanic.csv')
```

- **Structural Dataset Diagnosis:** Total DataFrame configuration, raw rows runtime configuration array layout representation dynamic checking.

```
dataset
```

- **Identifying Missing Gaps:** Initial missing variables evaluation metrics, raw features runtime inspection ebong null components pattern check.

Class 02: Missing Data Diagnosis & Descriptive Statistical Imputations

Core Notebook Reference: ML_Class03.ipynb (Titanic Dataset Processing)

- **Quantifying Gaps (Null Value Counting):** Real-time parameters runtime column checking logic layout mapping feature column-wise sum validation matrix representation.

```
dataset.isnull().sum()
```

- **Exploratory Visual Data Assessment:** Matplotlib lines structures configurations framework setup single parameter features representation lines plot configuration.

```
plt.plot(dataset['Age'])
```

- **Descriptive Imputation Workflows:** Data validation standard rules adjustments mean calculations parameters replace setup:
  - **Mean Imputation:** Continuous values average format calculations runtime parameters updates execution filter setup.

```
dataset['Agemean'] = dataset['Age'].fillna(dataset['Age'].mean())
```

  - **Mode Imputation:** Most frequent repeating parameter code blocks alignment adjustments execution setups.

```
dataset['Agemode'] = dataset['Age'].fillna(dataset['Age'].mode()[0])
```

  - **Median Imputation:** Middle numeric index scale features alignment setups updates mapping sequence configuration.

```
dataset['Agemedian'] = dataset['Age'].fillna(dataset['Age'].median())
```

- **Imputation Verification Check:** Dynamic structural parameters logic validations code runtime counters update check optimization array values update verification execution.

```
dataset.isnull().sum()
```

Class 03: Random Sample Imputation, Feature Mapping & Continuous Regression Workflows

Core Notebook References: ML_Class03.ipynb & Working_With_Study_Hours_Dataset.ipynb

- **Advanced Imputation (Random Sample Imputation):** Data frame rows structure variations logic mapping values random sample distribution extraction logic optimization processing.

```
random_sample = dataset['Age'].dropna().sample(dataset['Age'].isnull().sum(), random_state=0)
```

- **Feature Matrix & Vector Isolation:** Multi-dimensional dataframe parameters alignment logic matrices tracking independent spaces adjustments validation split baseline settings.
- **Row Slicing & Specific Node Extraction:** Data variables configuration conditional brackets query index positioning tracking parameters slice checks updates representation.

```
dataset.head(10)
```

- **Continuous Target Analysis:** Mathematical relationship mapping frameworks execution where scale independent feature nodes (such as `Hours of Study`) dynamically influence single target metric continuous configurations (such as `Exam Score`).
- **Linear Metric Correlation Profiling:** Feature parameter directional dependency checks multi-column matrix computations:

```
dataset.corr()
```

- **Statistical Heatmap Visualization:** Multi-variable correlation grids matrix visual validation parameters plot structures mapping configurations utilizing Seaborn library modules processing.

```
import seaborn as sns
sns.heatmap(dataset.corr())
# Extension: sns.heatmap(dataset.corr(), annot=True)
```

Class 04: Advanced Feature Engineering, Scoring & Algorithmic Feature Selection

Core Notebook Reference: ML_With_Python_Class04.ipynb (Mobile Dataset)

- **Variance Discrepancy Auditing:** Model features data variance range skews checks structural evaluations configurations adjustment metrics (e.g., wide continuous values such as `ram` or `battery_power` versus miniature configurations parameters scaling like `clock_speed`).
- **Statistical Parameter Selection (SelectKBest):** Best predictive categorical scores evaluation targets tracking matrix layout mapping using Chi-Square (`chi2`) algorithm scoring framework system rules setups.

```
from sklearn.feature_selection import SelectKBest, chi2
bestfeatures = SelectKBest(score_func=chi2, k=10)
fit = bestfeatures.fit(X, y)
```

- **Tree-Based Feature Importance Isolation:** Model ensemble classifiers fitting loops tracking computations Gini importance parameters coefficients metrics printing layouts mapping visualization plots generation.

```
from sklearn.ensemble import ExtraTreesClassifier
model = ExtraTreesClassifier()
model.fit(X, y)
print(model.feature_importances_)
```

- **Multi-Collinearity Redundancy Pruning:** High correlation target properties matrix tracking system filtering models parameters dropped logic selection array configuration mapping structure layouts tracking.

Class 05: Data Transformation, Categorical Label Encoding & Validation Splits

Core Notebook Reference: ML_With_Python_Class05_ipynb.ipynb (Survey Dataset Processing)

- **Survey Qualitative Dataset Handling:** Raw unstructured non-numeric multi-class items textual response variables mapping arrays formatting system structural checking properties tracking.
- **Ordinal Label Encoding:** Text attributes or string tracking items encoding transformations matrix into machine-readable continuous sequencing integer values parameters conversion structures execution tracking setups.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
dataset['column'] = le.fit_transform(dataset['column'])
```

- **Matrix Segmentation Strategy (X and y Split):** Features columns slice adjustments processing configurations parsing tables rows layout array splits rules configuration system settings logic structures.

```
X = dataset.iloc[:, :-1]
y = dataset.iloc[:, -1]
```

- **Stochastic Validation Protocols (Train-Test Split):** Unbiased performance check execution test matrix splits validation checks configuration arrays array segments parameters setups formatting logic properties setup models tracking bounds.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)
```

Class 06: Supervised Classifiers Suite & Model Performance Benchmarking

Core Notebook Reference: ML_With_Python_Class06_ipynb.ipynb (Model Comparison Framework)

- **Multi-Algorithmic Supervised Estimators Training:** Diverse mathematical operations boundaries structures fitting execution array processing optimization model options options validation:
  - **Tree Architecture Estimators:** `DecisionTreeClassifier`, `RandomForestClassifier`, `ExtraTreesClassifier`
  - **Sequential Boosting Architecture Models:** `GradientBoostingClassifier`, `AdaBoostClassifier`
  - **Distance & Margin Classifiers Suite:** `KNeighborsClassifier`, `SVC` (Support Vector Classifier)
  - **Linear Boundaries & Probabilistic Estimates:** `GaussianNB` (Naive Bayes), `RidgeClassifier`, `LinearDiscriminantAnalysis`, `SGDClassifier`
- **Model Loop Pipeline & Scoring Metrics Tracking:** Automated iterative classification checking arrays model validation script pipeline variables prediction accuracy tracking scoring updates computations.

```
from sklearn.metrics import accuracy_score
ypred = model.predict(X_test)
accuracy_score(y_test, ypred)
```

- **Model Benchmark Compilation:** Model evaluation outcome ranking structures parameters processing dictionary arrays configuration sorting updates output printing systems configurations execution setups tracking.

```
# Sorted dictionary ranking tracking outputs based on accurate scores tracking
logic metrics
for model, acc in sorted(results.items(), key=lambda x: x[1], reverse=True):
    print(f"{model}: {acc:.4f}")
```

Class 07: End-to-End Predictive Machine Learning Pipeline Consolidation

Core Notebook Reference: ML_With_Python_Class07.ipynb (Consolidated Workflow Workspace)

- **Full Blueprint Synthesis:** Processing arrays adjustments configurations, multi-column value checks validations processing splits array configurations training workflow consolidation implementation using a completely new dataset structure.
- **Complex Feature Correlation Metrics Execution:** Parallel feature data grid plotting tracking mapping evaluations architectures cross-feature configurations layout data check processing validations.
- **Multi-Column Formatting Pipelines Scaling:** Enforcing clean data conversion arrays processing pipelines concurrently transformations cross multiple tracking parameters tracking categories label setups.
- **Unified Pipeline Performance Evaluation Matrix:** Running dynamic test validations array sets optimization comparison reports overall multi-model target metrics outcome sheet output.